

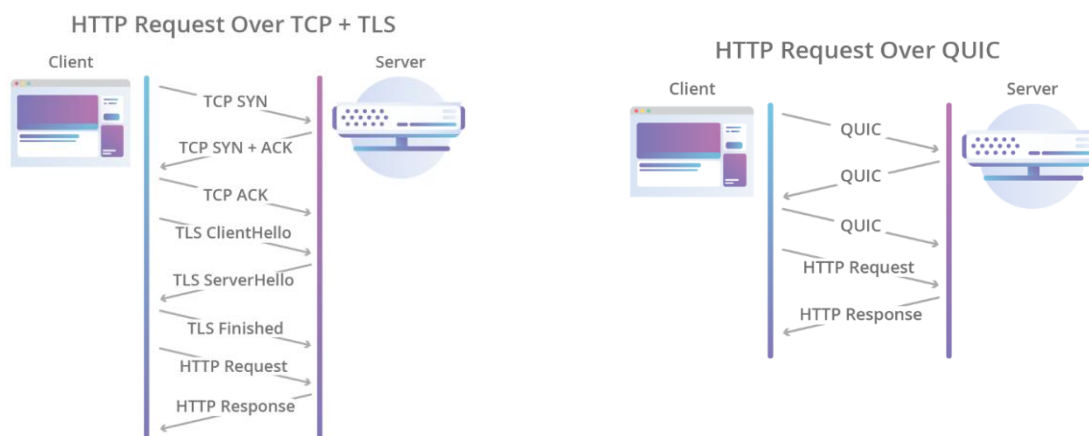
QUIC – Technical Overview

HPE CPP Week-1 Report

Name: Vansh Chani

QUIC (Quick UDP Internet Connections) is a new encrypted-by-default Internet transport protocol, that provides a few improvements designed to accelerate HTTP traffic as well as make it more secure, with the intended goal of eventually replacing TCP and TLS on the web.

The initial QUIC handshake combines the typical three-way handshake that you get with TCP, with the TLS 1.3 handshake, which provides authentication of the end-points as well as negotiation of cryptographic parameters. QUIC replaces the TLS record layer with its own framing format, while keeping the same TLS handshake messages.



Design of QUIC

QUIC is built on top of UDP (User Datagram Protocol) instead of TCP.

Why UDP?

1. Avoids Kernel Restrictions

TCP is implemented inside the operating system kernel.

This means:

- Updating TCP requires OS updates.
- Innovation is slow.
- Developers have limited flexibility.

Since UDP is minimal and simple, QUIC can implement its own transport logic in user space, bypassing kernel-level constraints.

2. Can Be Implemented in User Space

Because UDP just sends raw datagrams:

- QUIC runs entirely in application layer (e.g., inside Chrome, servers like nginx).
- Developers can push updates without waiting for OS upgrades.
- Faster experimentation and deployment.

This is one of the biggest reasons why companies like Google were able to evolve QUIC quickly.

Issues with UDP and how QUIC manages those:

UDP does not guarantee:

- Packet delivery
- Order
- Duplicate prevention

QUIC adds:

- Packet numbering
- Acknowledgment (ACK) frames
- Retransmission of lost packets
- Loss detection mechanisms

Built-in Encryption:

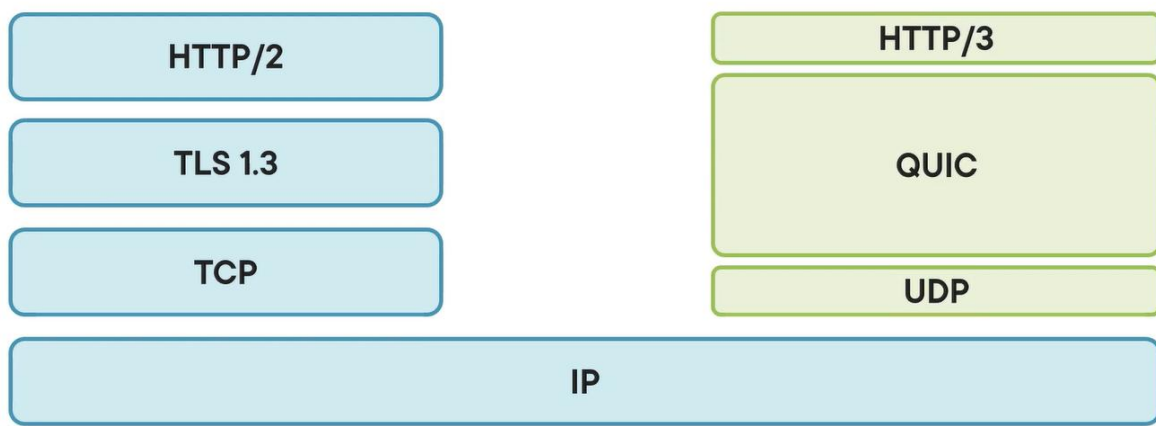
QUIC integrates **TLS 1.3** directly.

There is:

- No unencrypted QUIC
- No separate TLS handshake

Encryption is mandatory.

The QUIC Protocol Stack

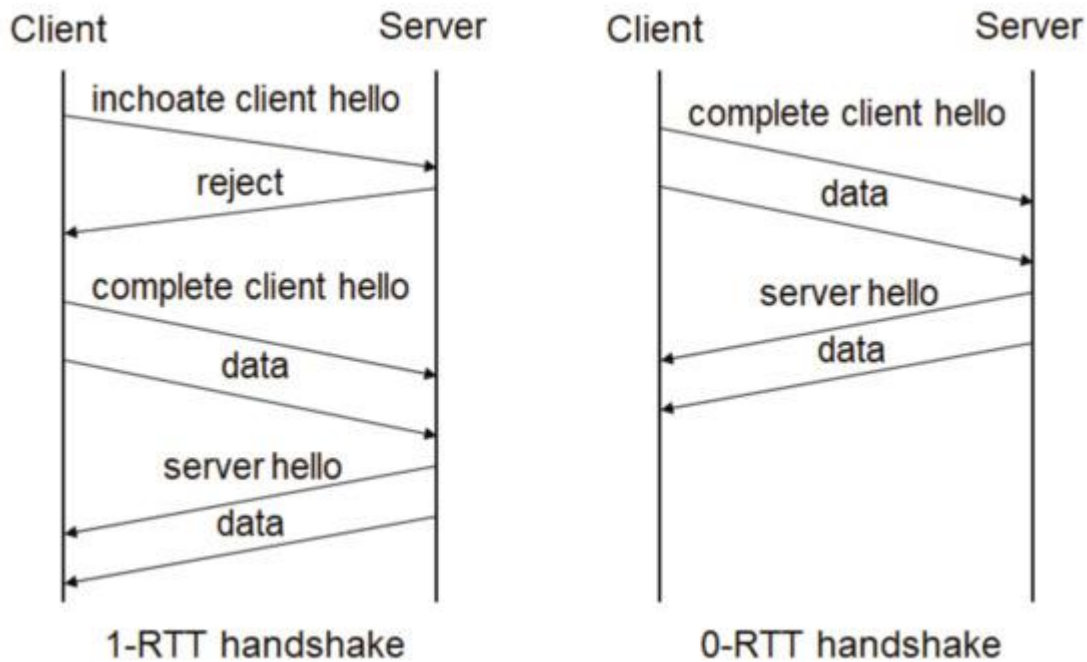


The left stack shows the traditional web setup: HTTP/2 runs on top of TLS 1.3 for security, which runs on TCP for reliable data delivery. TCP ensures packets arrive in order and handles congestion control. All of this sits on top of IP, which routes data across networks.

The right stack shows the modern approach with HTTP/3. Instead of TCP, it uses QUIC, which runs over UDP. QUIC combines security (like TLS), reliability, and congestion control into one protocol.

The main difference is that HTTP/3 replaces TCP + TLS with QUIC over UDP. This allows faster connections, better handling of multiple data streams, and improved performance.

The QUIC Handshake



1-RTT handshake → normal secure handshake

0-RTT handshake → client sends data immediately using previously saved session info

1-RTT:

If no previous session exists:

Client → Server:

- ClientHello (TLS)
- Transport parameters

Server → Client:

- ServerHello
- Keys
- Encrypted response

Data can start flowing after 1 RTT.

0-RTT:

If client has connected before:

Client immediately sends:

- Encrypted application data
- Using stored session keys

No waiting.

This drastically improves page load time.

How it happens:

Client first tries 0-RTT but if it fails then server sends "reject" and then it falls back to 1-RTT. If the connection was recent then the parameters match and the server accepts the data sent with the ClientHello itself.

Capturing QUIC with Wire shark:

1. Open Wireshark
2. Select your active network interface (Wi-Fi / Ethernet)
3. Click Start Capture
4. Apply filter `udp.port == 443` or `quic`
5. To generate QUIC traffic visit <https://cloudflare-quic.com>

No.	Time	Delta	Source	Destination	Protocol	Length	Info
467	8.504646100	0.00079	172.17.93.2	142.251.221.196	QUIC	1292	Initial, DCID=b256d8b5aa6629a3, PKN: 1, CRYPTO, PADDING, PING, PING, CRYPTO, PADDING, PING, PADDING, CRYPTO, CRYPTO, CRYPTO, PADDING, PING, CRYPTO, P...
479	8.520128000	0.01519	142.251.221.196	172.17.93.240	QUIC	82	Initial, SCID=f256d8b5aa6629a3, PKN: 1, ACK
480	8.521908800	0.00178	142.251.221.196	172.17.93.240	QUIC	1292	Initial, SCID=f256d8b5aa6629a3, PKN: 2, ACK, PADDING
481	8.521908800	0.00000	142.251.221.196	172.17.93.240	QUIC	1292	Initial, SCID=f256d8b5aa6629a3, PKN: 3, CRYPTO, PADDING
482	8.524513100	0.00260	172.17.93.2	142.251.221.196	QUIC	1292	Initial, DCID=f256d8b5aa6629a3, PKN: 3, ACK, PADDING
486	8.528099700	0.00358	142.251.221.196	172.17.93.240	QUIC	1292	Initial, SCID=f256d8b5aa6629a3, PKN: 4, CRYPTO, PADDING
494	8.530918000	0.00499	172.17.93.2	142.251.221.196	QUIC	1292	Initial, DCID=f256d8b5aa6629a3, PKN: 4, ACK, PADDING
496	8.538306600	0.00526	172.17.93.2	142.251.221.196	QUIC	79	Handshake, DCID=f256d8b5aa6629a3
502	8.539761800	0.00140	142.251.221.196	172.17.93.240	QUIC	823	Protected Payload (KP0)
514	8.545692300	0.00593	172.17.93.2	142.251.221.196	QUIC	82	Handshake, DCID=f256d8b5aa6629a3
516	8.554800600	0.00910	142.251.221.196	172.17.93.240	QUIC	1292	Handshake, SCID=f256d8b5aa6629a3
517	8.554800600	0.00000	142.251.221.196	172.17.93.240	QUIC	152	Protected Payload (KP0)
518	8.555463300	0.00066	172.17.93.2	142.251.221.196	QUIC	83	Handshake, DCID=f256d8b5aa6629a3
519	8.555808300	0.00054	172.17.93.2	142.251.221.196	QUIC	83	Handshake, DCID=f256d8b5aa6629a3
530	8.571411100	0.01560	172.17.93.2	142.251.221.196	QUIC	1292	Initial, DCID=9219996f56fa804, PKN: 1, CRYPTO, CRYPTO, PING, PADDING, CRYPTO, PING, PADDING, PING, PADDING, CRYPTO, CRYPTO, PADDING, CRYPTO, CRYPTO, ...
531	8.571627800	0.00021	172.17.93.2	142.251.221.196	QUIC	1292	Initial, DCID=9219996f56fa804, PKN: 2, PADDING, CRYPTO, PING, PADDING, CRYPTO, PING, PING, PING, CRYPTO, PING, PADDING, PING, PING, CRYPTO, CRYPTO, ...
532	8.572111500	0.00048	142.251.221.196	172.17.93.240	QUIC	1292	Handshake, SCID=f256d8b5aa6629a3
533	8.572111500	0.00000	142.251.221.196	172.17.93.240	QUIC	1292	Handshake, SCID=f256d8b5aa6629a3
536	8.573783500	0.00167	172.17.93.2	142.251.221.196	QUIC	85	Handshake, DCID=f256d8b5aa6629a3
537	8.574688600	0.00090	172.17.93.2	142.251.221.196	QUIC	86	Handshake, DCID=f256d8b5aa6629a3
546	8.581762200	0.00707	172.17.93.2	142.251.221.196	QUIC	125	Handshake, DCID=f256d8b5aa6629a3
547	8.582292900	0.00053	172.17.93.2	142.251.221.196	QUIC	116	Protected Payload (KP0), DCID=f256d8b5aa6629a3
548	8.582681800	0.00038	172.17.93.2	142.251.221.196	QUIC	71	Protected Payload (KP0), DCID=f256d8b5aa6629a3

```
▶ Frame 468: Packet, 1292 bytes on wire (10336 bits), 1292 bytes captured (10336 bits) on interface \Device\NPF_{EAB...
▶ Ethernet II, Src: WNC_0c:6c:f4 (40:1a:58:0c:6c:f4), Dst: HewlettPacka_ce:84:1a (68:b5:99:ce:84:1a)
▶ Internet Protocol Version 4, Src: 172.17.93.240, Dst: 142.251.221.196
▶ User Datagram Protocol, Src Port: 55071, Dst Port: 443
▶ QUIC IETF
  ▶ QUIC Connection information
    [Packet Length: 1250]
    1... .... = Header Form: Long Header (1)
    1.. .... = Fixed Bit: True
    ..00 .... = Packet Type: Initial (0)
    [... 00.. = Reserved: 0]
    [... ..01 = Packet Number Length: 2 bytes (1)]
    Version: 1 (0x00000001)
    Destination Connection ID Length: 8
    Destination Connection ID: b256d8b5aa6629a3
    Source Connection ID Length: 0
    Token Length: 0
    Length: 1232
    [Packet Number: 2]
    Payload [...]: 50a679807b5f3fb715ec07308cdf5912826caa384f876f8fa991f27149db45258470ef906ad3b6b6b224cb299ea8ae6d92c
  ▶ PING
    Frame Type: PING (0x0000000000000001)
  ▶ CRYPTO
    Frame Type: CRYPTO (0x0000000000000006)
```

```
  ▶ CRYPTO
    Frame Type: CRYPTO (0x0000000000000006)
    Offset: 950
    Length: 2
    Crypto Data
    ▶ [15 Reassembled QUIC CRYPTO Data Fragments (1782 bytes): #467(45), #468(732), #468(14), #468(84), #468(31), #4...
    ▶ TLSv1.3 Record Layer: Handshake Protocol: Client Hello
      Handshake Protocol: Client Hello (last fragment)
      ▶ [3 Reassembled Handshake Fragments (1801 bytes): #467(15), #467(4), #468(1782)]
      ▶ Handshake Protocol: Client Hello
        Handshake Type: Client Hello (1)
        Length: 1797
        ▶ Version: TLS 1.2 (0x0303)
          ▶ [Expert Info (Chat/Deprecated): This legacy_version field MUST be ignored. The supported_versions exte...
            [This legacy_version field MUST be ignored. The supported_versions extension is present and MUST be...
            [Severity level: Chat]
            [Group: Deprecated]
          Random: cd493387c7b3dd877faa35d39193173d82c51f49f3b74fde1a77e4bfd1af58a8
          Session ID Length: 0
          Cipher Suites Length: 6
```

```
▼ Extension: server_name (len=19) name=www.google.com
  Type: server_name (0)
  Length: 19
  ▼ Server Name Indication extension
    Server Name list length: 17
    Server Name Type: host_name (0)
    Server Name length: 14
    Server Name: www.google.com
```

Some Distinguishing Features between QUIC traffic and UDP Noise

1. Most QUIC traffic (especially HTTP/3 over QUIC) runs on: **UDP port 443** So if you see: UDP → Port 443 It's likely QUIC.
2. Even though QUIC encrypts most of its payload, its **header format is structured**.
 - a. QUIC packets always contain:
 - b. Header Form bit (Long or Short)
 - c. Fixed bit (The second-most-significant bit of the first byte is always set to 1.)
 - d. Version field (in long headers)
 - e. Connection ID
 - f. Packet number (encrypted but format predictable)

3. Long Header During Handshake:

During connection setup, QUIC uses a **Long Header** that contains:

- a. Version number (e.g., 0x00000001)
- b. Destination Connection ID
- c. Source Connection ID

If the Version field matches known QUIC versions → strong indicator it's QUIC.

4. Identification based on Traffic Behaviour:

QUIC has recognizable traffic behaviour:

- Initial packets ~1200 bytes (minimum size requirement)
- TLS 1.3 handshake inside
- Multiple streams
- Encrypted frames
- Regular ACK frames

Main QoS and Policing Methods Used with QUIC

1. DSCP (DiffServ)

Packets can be marked with priority values in the IP header. Routers then prioritize traffic based on those markings.

2. Rate Limiting and Policing

Token bucket or leaky bucket algorithms limit bandwidth per:

- User
- IP address
- Port (commonly UDP 443)
- Flow (5-tuple)

3. Flow-Based Queuing

Routers apply fairness algorithms (e.g., WFQ, FQ) using the 5-tuple:

- Source IP
- Destination IP
- Source Port
- Destination Port
- Protocol (UDP)

Each QUIC connection appears as one UDP flow.

4. Active Queue Management (AQM)

Mechanisms like RED or CoDel manage congestion and reduce latency. These work well because QUIC responds to congestion signals.

5. ECN (Explicit Congestion Notification)

QUIC supports ECN, allowing networks to signal congestion without dropping packets.

6. DPI-Based Policies (Limited)

During handshake, some metadata (like server name) may be visible. Networks can classify traffic at that stage, but visibility is decreasing due to encryption improvements.

7. Blocking or Throttling UDP 443

Some networks block or limit UDP 443 to control QUIC traffic, forcing fallback to TCP.

Summary Table

Method	Works for QUIC?	Granularity
DSCP	Yes	Per packet
Token bucket policing	Yes	Per flow / per user
WFQ / FQ	Yes	Per 5-tuple flow
DPI	Limited	Domain-level
ECN	Yes	Congestion-based
Per-stream QoS	No (not visible to network)	Internal only